Contents

Docker — Interview Questions & Answers **1**

Docker — Interview Questions & Answers

Q1: What is the difference between a Docker image and a container? > **Image:** Read-only template with layers. Blueprint. Stored in registry. > **Container:** Running instance of an image. Has a writable layer on top. Ephemeral (data lost on removal unless using volumes).

Q2: Explain Docker layer caching and how to optimize it. > Each instruction in Dockerfile creates a layer. Cache is invalidated when a layer changes — all subsequent layers rebuild. > **Optimization:** > 1. Put instructions that change rarely at the top (FROM, RUN apt-get) > 2. Copy dependency files before source code:

```
COPY requirements.txt .    # cache this layer
RUN pip install -r requirements.txt
COPY . .                   # only invalidates from here
```

3. Use `--mount=type=cache` in BuildKit

Q3: What is a multi-stage build and why is it important? > Allows using multiple FROM instructions. Build artifacts compiled in one stage (with all build tools) are copied to a minimal final image. Result: production image has no build tools = smaller attack surface, faster pull times. > Example: 400MB Go builder → 8MB distroless final image.

Q4: How does Docker networking work? What is the difference between bridge and host networking? > **Bridge:** Default. Creates virtual network interface on host (`docker0`). Containers get private IPs, NAT for outbound. Port mapping creates iptables DNAT rules. > **Host:** Container shares host's network namespace directly. No NAT, no port mapping needed. Best performance. Security risk — container can see all host interfaces.

Q5: How do you handle secrets in Docker? > **DON'T:** Bake into image (ENV, COPY of secret files) — visible in image history > **DO:** > 1. Pass at runtime: `docker run -e SECRET=$(vault kv get -field=value secret/myapp)` > 2. Docker secrets (Swarm): `docker secret create mysecret file` → mounted at `/run/secrets/` > 3. Build-time secrets with BuildKit: `--secret id=npmrc,src=~/.npmrc` > 4. Kubernetes secrets / Vault agent sidecar in production

Q6: Container A cannot reach Container B. How do you debug?

```
# 1. Are they on the same network?
docker inspect A | grep Networks
docker inspect B | grep Networks

# 2. Can A ping B?
docker exec A ping B

# 3. Check port — is B listening?
docker exec B ss -tlnp

# 4. Test TCP connection
docker exec A nc -zv B 8080

# 5. Check container IPs
docker inspect -f '{{.NetworkSettings.IPAddress}}' B

# 6. If different networks, connect to same network:
docker network connect mynet A
```

Q7: How would you reduce a Docker image from 800MB to <100MB? > 1. Use minimal base: `FROM node:20-alpine` (50MB) vs `FROM node:20` (1GB) > 2. Multi-stage build — exclude node_modules from final image if possible > 3. Combine RUN commands to reduce layers > 4. Remove apt cache: `rm -rf`

`/var/lib/apt/lists/*` > 5. Use `--no-install-recommends` for apt > 6. Delete build artifacts after compilation > 7. `.dockerignore` to exclude test files, docs > 8. Consider distroless base

Q8: What is the `ENTRYPOINT` vs `CMD`? > **ENTRYPOINT:** The executable. Not easily overridden (requires `--entrypoint` flag). > **CMD:** Default arguments to ENTRYPOINT, easily overridden at runtime.

```
ENTRYPOINT ["/app/server"]   # always run server
CMD ["--port", "8080"]       # default port, override with: docker run img --port 9090
```

Exec form `["cmd"]` preferred over shell form `cmd` (no shell wrapping, signals go directly to process).

Q9: How does `COPY --from=builder` work in multi-stage builds? > Copies files from another named build stage. The builder stage is not included in the final image. You can also copy from external images: `COPY --from=gcr.io/distroless/static /busybox /busybox`.

Q10: Explain Docker's `restart` policies. | Policy | Behavior | |——–|———| | no | Never restart (default) | | always | Always restart, including on daemon start | | `on-failure` | Restart if exit code non-zero | | `unless-stopped` | Always restart unless manually stopped | > Use `unless-stopped` for production services.

Q11: How do you perform a zero-downtime deployment with Docker Compose?

```
# Scale up new version
docker-compose up -d --scale api=2 --no-recreate

# Once healthy, remove old containers
docker-compose up -d --remove-orphans

# Or use rolling updates with Swarm:
docker service update --image myapp:v2 --update-parallelism 1 \
  --update-delay 30s myapp_service
```

Q12: How do you monitor container resource usage?

```
docker stats                          # live CPU/memory/network/disk
docker stats --no-stream --format \
  "table {{.Container}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# cAdvisor for metrics export to Prometheus
docker run -d --name cadvisor \
  -v /:/rootfs:ro \
  -v /var/run:/var/run:ro \
  -v /sys:/sys:ro \
  -v /var/lib/docker:/var/lib/docker:ro \
  -p 8080:8080 \
  gcr.io/cadvisor/cadvisor:latest
```

Q13: What is a distroless image? When should you use it? > Distroless images (by Google) contain only the application and its runtime dependencies. No shell, no package manager, no OS utilities. > **Benefits:** Tiny size, minimal CVEs, reduced attack surface (no shell = harder to exploit) > **Drawback:** Harder to debug (no exec with bash) > **Use for:** Production containers. Have a debug variant: `FROM gcr.io/distroless/java17-debian12:debug`

Q14: How do you implement health checks?

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

```
docker ps   # shows Up (healthy) or Up (unhealthy)
docker inspect --format='{{.State.Health.Status}}' container
docker inspect --format='{{range .State.Health.Log}}{{.Output}}{{end}}' container
```